BigQuery to MongoDB Synchronization Pipeline

1. Introduction

1.1. Purpose & Business Goal

This document provides a comprehensive architectural and operational guide for the hourly data synchronization pipeline between Google BigQuery and MongoDB Atlas.

The primary business goal of this pipeline is to hydrate our transactional application database (MongoDB) with data mastered in our analytical data warehouse (BigQuery). This provides application services with low-latency access to fresh, relevant data without incurring the performance or cost penalties of querying a data warehouse for operational workloads. The architecture is designed to be robust, scalable, and maintainable, supporting the specific data consistency models required by our various application services.

1.2. Scope

This document covers:

- The end-to-end architecture of the pipeline.
- The core Google Cloud and MongoDB components.
- Detailed descriptions of the three data synchronization patterns.
- Security, networking, and operational procedures.
- Implementation guidelines for the Java-based processing service.
- Monitoring, alerting, and CI/CD best practices.

1.3. Audience

This document is intended for software engineers, data engineers, DevOps specialists, and architects involved in the development, deployment, and maintenance of this pipeline.

2. Architecture Overview

2.1. Overview

The BigQuery-to-MongoDB Synchronization Pipeline is a scalable, self-contained system built entirely on **Google Cloud Dataflow**. It uses Dataflow's native scheduling capabilities to trigger large-scale, parallel data synchronization jobs on an hourly basis. All orchestration logic, including concurrency control and post-load operations, is managed within the Dataflow pipeline's driver program.

The architecture is built around the specific needs of three core collections:

1. `slot` **Collection:** Synchronized using a **Full Replacement** strategy to ensure it is an exact mirror of the source, including deletions.
2. `owner` **Collection:** Synchronized using an **Incremental Append** strategy for immutable, time-series data.
3. `release` **Collections:** Managed via an **Application-Managed & Unified View** strategy to handle complex application state while providing a simple interface to consumers.

The system is designed with security and reliability as first-class citizens, utilizing **VPC Peering** for private networking, **HashiCorp Vault** for credentials, and a full suite of **Cloud Monitoring** and alerting tools for Dataflow.

2.2. High-Level Architecture Diagram

This diagram shows the simplified, self-contained nature of the architecture. Dataflow's internal scheduler triggers a job for a specific collection, which then executes the appropriate pattern from reading the source to writing to the destination.

3. Core Architecture Deep Dive

This section details the primary components and their roles within the pipeline.

3.1. Orchestration & Processing Layer: Dataflow

In this architecture, **Dataflow is responsible for both orchestration and processing**.

- **Dataflow Scheduler:** The pipeline is triggered using Dataflow's built-in scheduler. This replaces the need for Cloud Scheduler and Pub/Sub. Each scheduled job is configured with the specific parameters (strategy, source table, destination collection) required for its task.
- **Dataflow Pipeline:** A fully managed, massively parallel data processing service for executing Apache Beam pipelines. It was chosen for its power and scalability, ensuring the pipeline can handle future data growth and increasing transformation complexity without requiring a re-architecture. It automatically provisions and manages the necessary worker resources for each job.
- **Dataflow Driver Program:** The main Java class that defines the pipeline is also responsible for all orchestration logic. This includes pre-flight checks like concurrency control and post-flight operations like the atomic collection swap in MongoDB.

3.2. Data & Storage Layer

- **BigQuery:** The authoritative source of truth for the data being synchronized. The pipeline reads from pre-defined, and potentially optimized, BigQuery views.

- **MongoDB Atlas:** The target operational database serving live application traffic.

3.3. Supporting Services

- **Artifact Registry:** A private Docker container registry used to store the versioned container images for our Dataflow Flex Templates.
- **Cloud Logging & Monitoring:** Provides centralized logging, metrics, dashboards, and alerting for all pipeline components.

3.4. Decision Matrix: Dataflow vs. Cloud Run

During the architectural design phase, both Dataflow and Cloud Run were considered. **Dataflow was selected** to prioritize long-term scalability and processing power. This matrix documents the trade-offs and rationale for this strategic decision.

Factor	Dataflow	Cloud Run	Justification & Rationale
Scalability	**Exceptional.** Massively parallel processing engine designed for terabyte-scale jobs. Automatically scales worker nodes.	**Good.** Scales based on concurrent requests, but processing is limited to a single container instance per job.	Dataflow was chosen to future-proof the architecture. It ensures that significant growth in data volume will not degrade performance or require a redesign. It is a strategic platform choice.
Complexity & Capability	**High.** Requires specialized knowledge of the Apache Beam SDK, but provides a rich framework for complex	**Low.** Uses standard container-based development, but is less suited for complex, large-scale data	By adopting Dataflow now, we standardize on a powerful ETL platform. This allows us to handle potentially

	transformations, windowing, and aggregations.	transformation tasks.	complex future requirements (e.g., joins, enrichments) within the same framework.
Use Case Fit	**Excellent.** Designed specifically for large-scale data movement and transformation between systems like BigQuery and external databases.	**Good.** Well-suited for simple, short-lived jobs, but can be limited by single-instance memory and processing power for larger datasets.	Dataflow is the purpose-built tool for this class of problem. Its I/O connectors are highly optimized for sources like BigQuery.
Cost Model	**Higher.** Higher base cost due to the overhead of spinning up a worker cluster. Billed per vCPU/memory-hour for workers.	**Lower.** Pay-per-use with millisecond billing. No cost between runs.	The higher cost of Dataflow is accepted as a strategic investment in scalability and capability. For very large jobs, Dataflow's efficiency can make it more cost-effective than a long-running Cloud Run instance.

Conclusion: Dataflow was selected as a strategic platform decision. It provides the long-term scalability and powerful data processing capabilities required for a core data integration pipeline, justifying the higher initial complexity and cost compared to Cloud Run.

4. Security & Networking

4.1. Network Isolation: VPC Peering

To ensure that data never traverses the public internet, a **VPC Peering** connection is established between our Google Cloud VPC and the MongoDB Atlas VPC. Dataflow workers will be configured to launch within our peered VPC.

- **Benefits:**
  - **Security:** Eliminates public internet exposure, reducing the attack surface.
  - **Performance:** Provides lower, more predictable network latency compared to public connections.
- **Implementation:** Firewall rules are configured on both sides to only allow traffic from the Dataflow worker IP range to the MongoDB cluster on the required ports (e.g., 27017).

4.2. Credentials Management: HashiCorp Vault

All sensitive information, especially the MongoDB Atlas connection string, is stored and managed by our central **HashiCorp Vault** instance.

- **Benefits:**
  - **Centralized Secrets:** Provides a single, secure source of truth for credentials across all our applications.
  - **Dynamic Secrets:** Vault can generate dynamic, just-in-time credentials for MongoDB, drastically reducing the risk of a leaked static password.
  - **Fine-Grained Access Control:** Access to secrets is controlled by Vault's flexible policy (ACL) system.
  - **Auditing:** All access to secrets is centrally audited and logged in Vault.
- **Implementation:** The Dataflow pipeline will authenticate to Vault using the **GCP Auth Method**. The service account associated with the Dataflow job will be authorized to log in to Vault and retrieve a token. This token will be associated with a Vault policy that grants read access to the specific secret path for the MongoDB connection string. This logic is encapsulated within the pipeline's driver program.

4.3. Principle of Least Privilege: IAM Roles

The Dataflow job operates under a dedicated IAM service account with the minimum set of permissions.

- **Dataflow Worker Service Account:**
  - `roles/dataflow.admin`: Required to allow the driver program to manage the pipeline's lifecycle.

- `roles/dataflow.worker`: Core permissions for a Dataflow worker to operate.
- `roles/bigquery.dataViewer` & `roles/bigquery.jobUser`: To read from BigQuery.
- `roles/iam.serviceAccountTokenCreator`: (For Vault Auth) To authenticate to Vault.
- `roles/logging.logWriter`: To write logs.
- `roles/datastore.user`: (For Concurrency Control) To access Firestore.

5. Data Models & Synchronization Philosophy

A core principle of this architecture is **clear data ownership**. The complexity of a data pipeline is often a direct result of ambiguity in the data model. To simplify our pipeline, we explicitly separate data mastered in BigQuery from data mastered by our application.

Collection Name	Master Data Source	Pipeline Interaction Pattern	Description
`slot`	**BigQuery**	**Full Replacement**. The pipeline owns and overwrites this collection.	A complete mirror of the source `slot` data, where all changes including deletions are synced.
`owner`	**BigQuery**	**Incremental Append**. The pipeline only adds new records.	An immutable, growing log of owner events, resilient to late-arriving data.
`releases_synced`	**BigQuery**	**Full Replacement**. The pipeline owns and overwrites this collection.	Contains only the final, terminal states of a `release`. It is a read-only mirror of the source.

`release_workflows`	**Application**	**None**. The pipeline never touches this collection.	Contains the in-progress states of a `release`. It is managed exclusively by the application.

The `release` use case is handled by the "Application-Managed & Unified View" strategy, which involves two collections and is detailed in Section 9.

6. Concurrency, Scaling, and State Management

To ensure the pipeline is robust and does not suffer from race conditions or cascading failures as data volumes grow, the following controls are mandated.

6.1. Problem: Concurrent Executions

If an hourly job's runtime exceeds 60 minutes, the Dataflow scheduler will trigger the launch of a new job while the first is still running. This can lead to data corruption.

6.2. Solution: Lease-Based Concurrency Control

The pipeline implements a locking mechanism using Firestore to ensure only one instance of a specific job can run at a time. This lock is managed by the **Dataflow driver program**.

Implementation Steps:

1. **Acquire Lock:** As the very first step in the `main` method of the driver program, the pipeline attempts to create a document in a dedicated Firestore collection (e.g., `_pipeline_locks/{collectionName}`).
2. **Check Lock:** If the document is created successfully, the job has acquired the lock and can proceed to build and run the Beam pipeline. If the document already exists, another job is running, and the driver program logs a message and exits with a success status, preventing a pipeline run.
3. **Release Lock:** The lock document is deleted in a `finally` block within the `main` method, ensuring it is released even if the pipeline fails.

7. Collection Sync Strategy: `slot` (Full Replacement)

This strategy ensures the `slot` collection is a perfect, point-in-time mirror of its source BigQuery view. It is the most robust method for synchronizing data that includes additions,

updates, and deletions.

7.1. Conceptual Overview

The strategy is analogous to a blue-green deployment for data. A new version of the collection is built in the background (the "staging" collection). Once it is complete and verified, traffic is atomically switched to it. This process is seamless to the application and its users, guaranteeing zero downtime.

7.2. Advantages

- **Handles Deletions Natively:** Any record not present in the source view is implicitly "deleted" because it won't be in the new collection.
- **Transactional Consistency:** The application will never see a partially loaded or inconsistent dataset.
- **Zero Downtime:** The `renameCollection` command is an atomic metadata operation, making the switch instantaneous.
- **Instant Rollback Path:** The old collection is preserved temporarily, allowing for a quick rollback by simply renaming it back.

7.3. Failure Handling & Transactionality

The entire process, including the final swap and cleanup, is managed by the Dataflow driver program.

- **Driver Program's Role:**
  a. Acquire concurrency lock for the `slot` collection.
  b. Perform pre-flight checks and orphan cleanup for any old `slot_staging` or `slot_old` collections.
  c. Create a new staging collection and apply indexes.
  d. Build and run the Beam pipeline to populate the staging collection, waiting for it to complete (`pipeline.run().waitUntilFinish()`).
  e. If the pipeline succeeds, perform the atomic swap and cleanup.
  f. Release concurrency lock in a `finally` block.

8. Collection Sync Strategy: `owner` (Incremental Append)

This strategy is a robust and efficient method for synchronizing the `owner` collection, which is an immutable log of events. It is designed to be resilient to late-arriving data.

8.1. Conceptual Overview

Instead of relying on event timestamps, this strategy processes data based on when it was ingested into BigQuery. By re-processing a rolling window of the last few days, the pipeline can correctly handle events that arrive late. The use of `upsert` ensures that re-processing data is idempotent and does not create duplicates.

8.2. Prerequisites

The source `owner` table in BigQuery **must** be partitioned by ingestion time (e.g., using the `_PARTITIONTIME` pseudo-column).

9. Collection Sync Strategy: `release` (Application-Managed & Unified View)

The `release` data model is the most sophisticated, involving two distinct collections (`releases_synced` and `release_workflows`) and a materialized view (`releases_unified_view`) to provide a simple, unified interface for application consumers.

9.1. Conceptual Overview: The Problem

Forcing every application to perform a dual-query to get a complete view of a `release` is inefficient and error-prone. It violates the DRY principle and distributes complex business logic across the system.

9.2. Solution: MongoDB Materialized View

The pipeline takes on the responsibility of creating a unified, pre-joined view of the data *within* MongoDB. Applications can then query a single, simple collection.

Implementation:

1. **Dataflow Syncs Data:** A Dataflow pipeline runs the "Full Replacement" job to update the `releases_synced` collection as normal.
2. **Driver Program Triggers Aggregation:** After the `waitUntilFinish()` call returns successfully, the **driver program** for the `releases_synced` job executes the MongoDB Aggregation Pipeline as its final step before releasing the lock.
3. **Aggregation Logic:** The aggregation uses `$lookup` and `$merge` to write the combined results into the `releases_unified_view` collection.

Result: Applications query the simple, de-normalized `releases_unified_view`, removing complex logic from their codebase.

10. Technical Implementation (Apache Beam)

This section provides implementation details and code examples using the Apache Beam SDK for Java.

10.1. Dependencies (pom.xml)

```
<dependencies>
    <!-- Beam SDK for Dataflow Runner -->
    <dependency>
        <groupId>org.apache.beam</groupId>
        <artifactId>beam-runners-google-cloud-dataflow-java</artifactId>
        <version>2.55.1</version>
    </dependency>
    <!-- Beam I/O for BigQuery -->
    <dependency>
        <groupId>org.apache.beam</groupId>
        <artifactId>beam-sdks-java-io-google-cloud-platform</artifactId>
        <version>2.55.1</version>
    </dependency>
    <!-- Beam I/O for MongoDB -->
    <dependency>
        <groupId>org.apache.beam</groupId>
        <artifactId>beam-sdks-java-io-mongodb</artifactId>
        <version>2.55.1</version>
    </dependency>
</dependencies>


```

10.2. Full Replacement Implementation (`slot` collection)

The core logic resides in the driver program, which orchestrates the Beam pipeline.

```
// Main class for the Dataflow job
public class FullReplacementPipeline {
    public static void main(String[] args) {
        // ... (Parse options, acquire lock) ...
        try {
            // 1. Pre-flight: Create staging collection, apply indexes, etc.
            String stagingCollectionName = mongoAdmin.prepareStagingCollection("slot");
            options.setStagingCollectionName(stagingCollectionName);

            // 2. Build and run the Beam pipeline
            Pipeline p = Pipeline.create(options);
            p.apply("ReadFromBigQuery", BigQueryIO.readTableRows().from(options.getSourceView()))
             .apply("TransformToDocument", ...)
             .apply("WriteToMongoStaging", MongoDbIO.write()
                .withCollection(options.getStagingCollectionName()));

            PipelineResult result = p.run().waitUntilFinish();

            // 3. Post-flight: Perform swap only if pipeline was successful
            if (result.getState() == PipelineResult.State.DONE) {
                mongoAdmin.performAtomicSwap("slot", stagingCollectionName);
            } else {
```

```
                throw new RuntimeException("Dataflow pipeline failed");
            }

        } finally {
            // 4. Always release the lock
            lockManager.releaseLock("slot");
        }
    }
}


```

10.3. Incremental Append Implementation (`owner` collection)

This pipeline uses a different query and a different write configuration.

```
// In the driver program for the 'owner' job:
// 1. Build a partition-based query
LocalDate startDate = LocalDate.now().minusDays(3);
String query = "SELECT * FROM `" + options.getSourceTable() + "` WHERE DATE(_PARTITIONTIME) >= '"
             + startDate.format(DateTimeFormatter.ofPattern("yyyy-MM-dd")) + "'";
options.setPartitionQuery(query);

// 2. Build and run the Beam pipeline
Pipeline p = Pipeline.create(options);
p.apply("ReadNewFromBigQuery", BigQueryIO.readTableRows().fromQuery(options.getPartitionQuery()))
 .apply("TransformToDocument", ...)
 .apply("UpsertToMongo", MongoDbIO.<Document>write()
    .withCollection("owner")
    .withUpdateConfiguration(
        MongoDbIO.Write.UpdateConfiguration.create()
            .withUpdateKey("_id").withIsUpsert(true)
    ));

p.run().waitUntilFinish();


```

11. Orchestration & Configuration

The entire pipeline is driven by Dataflow's native scheduling, with parameters passed directly to the job.

11.1. Dataflow Scheduler Configuration

A scheduled job is created directly in Dataflow for each synchronization task.

- **Schedule:** `0 * * * *` (At minute 0 of every hour).
- **Source:** A Dataflow Flex Template stored in GCS.
- **Parameters:** Key-value pairs passed as command-line arguments to the pipeline.

11.2. Example Job Parameters

For the `slot` collection (Full Replacement):

Parameter	Value
`--strategy`	`full_replacement`
`--sourceView`	`my-gcp-project.my_dataset.bq_slot_view`
`--targetCollection`	`slot`
`--runner`	`DataflowRunner`

For the `owner` collection (Incremental Append):

Parameter	Value
`--strategy`	`append_only`
`--sourceTable`	`my-gcp-project.my_dataset.bq_owner_table`
`--targetCollection`	`owner`
`--runner`	`DataflowRunner`

12. Dataflow Implementation Guide (Java)

12.1. Project Structure: Dataflow Flex Templates

The recommended approach is to package the pipeline as a **Dataflow Flex Template**. This involves creating a Docker image that contains the pipeline code, its dependencies, and a `main` class that acts as the driver program.

12.2. The Driver Program

The `main` method of the pipeline class is the central point of control. It is responsible for:

1. Parsing command-line options.
2. Acquiring and releasing the concurrency lock.
3. Performing pre-flight and post-flight operations (like the MongoDB collection swap).
4. Building and running the actual Apache Beam pipeline.

13. Monitoring, Logging, and Alerting

13.1. Structured Logging

The Apache Beam pipeline must use a logging library configured for structured JSON output to integrate with Cloud Logging.

13.2. Key Metrics & Dashboards

Create a custom dashboard in Cloud Monitoring with widgets for Dataflow jobs:

- **Job Status:** A chart showing job counts by status (Succeeded, Failed, Running).
- **Elapsed Time:** A time series chart of job runtimes.
- **Element Counts:** System lag and element counts for key pipeline steps.
- **Worker CPU & Memory Utilization:** To monitor resource usage and autoscaling.

13.3. Alerting Policies

Configure alerts in Cloud Monitoring based on Dataflow metrics:

Alert Name	Condition	Threshold	Notification Channel
Dataflow Job Failed	`dataflow.googleapis.com/job/is_failed`	> 0 for a specific job name pattern	PagerDuty / Slack
Anomalous Job Duration	`dataflow.googleapis.com/job/elapsed_time`	> 45 minutes	Slack

14. Operational Playbook (Runbook)

14.1. Alert: Dataflow Job Failed

- **Triage Steps:**
  a. Click the link in the alert to go to the **Dataflow Job Monitoring UI**.
  b. Inspect the **Job Error Log** and **Worker Logs** tabs for stack traces and error messages. Common causes:
     - `Permission Denied` on a source or sink.
     - `OutOfMemoryError` on workers (requires machine type adjustment).

    - Null pointers or data-related exceptions in the transformation logic.
  c. Check the "Parameters" tab to see the configuration the job was launched with.
- **Escalation:** Escalate to the Data Engineering team with a link to the failed Dataflow job.

15. CI/CD, Testing & Deployment

15.1. Source Control

The Apache Beam Java application code is stored in a Git repository.

15.2. Testing Strategy

- **Unit Tests (JUnit 5):** Use `TestPipeline` from the Beam SDK to test individual `PTransform`s and `DoFn`s in isolation.
- **Integration Tests (Testcontainers):** Use Testcontainers to test against real BigQuery and MongoDB instances.

15.3. CI/CD Pipeline (Jenkins)

A `Jenkinsfile` in the root of the repository defines the CI/CD pipeline.

1. **Trigger:** On push to the `main` branch.
2. **Stages:**
   - **Build & Test:** Compile the Java code and run all tests.
   - **Build Flex Template:**
     - Build the pipeline's "fat JAR".
     - Create the Docker image containing the JAR.
     - Use `gcloud dataflow flex-template build` to create the template spec file and upload it to a GCS bucket.
   - **Deploy Scheduled Job:** Use `gcloud dataflow jobs create my-scheduled-job --schedule="0 * * * *"` with the new template path, or update an existing scheduled job via Terraform.

15.4. Infrastructure as Code (IaC)

All GCP resources, including the **scheduled Dataflow jobs**, are defined declaratively using Terraform.

16. Data Quality & Reconciliation

16.1. Pre-Flight Checks

Before running the Beam pipeline, the driver program will perform basic validation on the source BigQuery view.

16.2. Post-Flight Reconciliation Report

Upon successful completion, the driver program will log a structured JSON reconciliation report.

Example Reconciliation Log:

```
{
  "severity": "INFO",
  "message": "Sync Reconciliation Report",
  "jobConfig": { "collectionName": "slot", "strategy": "full_replacement" },
  "metrics": {
    "sourceRowCount": 150234,
    "recordsWritten": 150234,
    "jobDurationSeconds": 1850
  }
}

```